

PROGRAMMING WITH PYTHON (202044504) - ASSIGNMENT 1

B.Tech. Information Technology - Semester V | Advanced Programming Questions with NAAC/OBE Mapping

Course Outcome Reference

CO	Course Outcome Statement
CO-1	Write, Test and Debug Python Programs
CO-2	Use functions and represent Compound data using Lists, Tuples and Dictionaries
CO-3	Text filtering with regular expressions in Python.
CO-4	Data manipulation and data visualization in Python.
CO-5	Create GUI applications in Python.

Instructions to Students

- Submit one Python file or a properly organized folder for each question as instructed by the faculty.
- Use Python 3.10 or above. External libraries may be used only when explicitly allowed in the question.
- Do not copy code from classmates, websites or AI tools without understanding; plagiarism will be handled as academic misconduct.
- Every program must include input validation, meaningful function names and comments for important logic blocks.
- Use the naming convention: Enrollment_AssignmentNo_QNo.py or Enrollment_AssignmentNo_QNo_folder.
- Mention compilation/run instructions, required packages, database schema or sample files wherever applicable.
- Test each program using sample tests, boundary tests and at least three additional self-created test cases.
- Do not submit solutions printed from this assignment document; submit only source code, screenshots if asked and test evidence.

Assignment 1 Questions

Q1. Campus Merit Analyzer using Compound Data Structures

Field	Details
Difficulty	Advanced University / Placement
CO Mapping	CO-2, CO-4
PO Mapping	PO1, PO2, PO5, PO10, PO12
PSO Mapping	PSO1, PSO2
Bloom Level	L4 Analyze
Skills Tested	Lists, tuples, dictionaries, sorting, hashing, data aggregation

Problem Statement: You are given records of students in the form enrollment, name, semester, CPI and a list of subject marks. Build a program that stores the records using lists, tuples and dictionaries. For each semester, print the top K students by CPI. If CPI is the same, prefer the student with higher average marks; if still tied, prefer lexicographically smaller enrollment number. Also print the subject-wise topper for every subject code.

Input Format: First line: n k m. Next n lines: enrollment name semester cpi followed by m integer marks. Subject codes are S1 to Sm.

Output Format: Semester-wise top K enrollments in sorted order as required, followed by subject-wise toppers.

Constraints: $1 \leq n \leq 100000$, $1 \leq k \leq 50$, $1 \leq m \leq 12$, $1 \leq \text{semester} \leq 8$, $0 \leq \text{marks} \leq 100$, $0.0 \leq \text{cpi} \leq 10.0$

Sample Input: 5 2 3

2201 Asha 5 8.90 90 82 75

2202 Bharat 5 8.90 88 95 70

2203 Chaitra 3 9.10 91 88 92

2204 Dev 5 7.80 76 80 85

2205 Esha 3 9.10 91 90 88

Sample Output: Semester 3: 2205 2203

Semester 5: 2202 2201

S1: 2203 2205

S2: 2202

S3: 2203

Sample Explanation: In semester 3, both students have CPI 9.10. Esha has higher average marks, so 2205 is listed first. For S1, two students share the highest mark.

Expected Time Complexity: $O(n*m + n \log n)$ or better using grouped sorting

Expected Space Complexity: $O(n*m)$

Q2. Optimized Password Audit with Pattern Constraints

Field	Details
Difficulty	Product-Based Company Level
CO Mapping	CO-1, CO-3
PO Mapping	PO1, PO2, PO5, PO8, PO12
PSO Mapping	PSO1, PSO2
Bloom Level	L5 Evaluate
Skills Tested	Strings, regular expressions, hashing/trie, validation, edge-case analysis

Problem Statement: Given n candidate passwords, classify each as STRONG, WEAK_LENGTH, WEAK_PATTERN or COMPROMISED. A strong password must contain at least one lowercase letter, uppercase letter, digit and one special symbol from $\$ \# @$. Its length must be between 6 and 12. It must not contain any dictionary word from a banned list as a contiguous substring, ignoring case. It must also not contain the same character more than 3 times consecutively.

Input Format: First line: b . Next b lines: banned words. Next line: n . Next n lines: one password per line.

Output Format: For each password, print password_index: classification.

Constraints: $1 \leq b \leq 10000$, total banned characters ≤ 200000 , $1 \leq n \leq 100000$, password length ≤ 100

Sample Input: 2

```
admin
college
5
Abc12@
adminA1@
AAAA1@b
short
Good9#
```

Sample Output: 1: STRONG

2: COMPROMISED

3: WEAK_PATTERN

4: WEAK_LENGTH

5: STRONG

Sample Explanation: The second password contains a banned word. The third has the same character repeated more than 3 times.

Expected Time Complexity: $O(\text{total password length} + \text{total banned length})$ preferred using Trie/Aho-Corasick; $O(n*b*L)$ is not acceptable

Expected Space Complexity: $O(\text{total banned length})$

Q3. Recursive Expression Engine with Memoization

Field	Details
Difficulty	FAANG Interview Level
CO Mapping	CO-1, CO-2
PO Mapping	PO1, PO2, PO3, PO5, PO12
PSO Mapping	PSO1, PSO2, PSO3
Bloom Level	L6 Create
Skills Tested	Recursion, stacks, dictionaries, parsing, memoization, cycle detection

Problem Statement: Implement a recursive expression evaluator for integer expressions containing non-negative integers, +, -, *, parentheses and variable references. Each variable may be defined by another expression. Detect cyclic variable dependencies and report CYCLE. Use memoization so that repeated variable evaluations are not recomputed.

Input Format: First line: v . Next v lines: variable_name = expression. Last line: expression_to_evaluate.

Output Format: Print the integer value, or CYCLE if cyclic dependency exists, or INVALID if expression syntax is invalid.

Constraints: $1 \leq v \leq 200000$, expression length total ≤ 500000 , values fit in signed 64-bit integer

Sample Input: 3

```
a = 2 + 3
b = a * 4
c = b - (a + 1)
c + 10
```

Sample Output: 24

Sample Explanation: $a = 5$, $b = 20$, $c = 14$, and $c + 10 = 24$.

Expected Time Complexity: $O(\text{total expression length})$

Expected Space Complexity: $O(\text{total expression length})$

Q4. Exception-Safe CSV Transaction Splitter

Field	Details
Difficulty	Advanced University / Industry
CO Mapping	CO-1, CO-4
PO Mapping	PO1, PO2, PO5, PO8, PO10, PO12
PSO Mapping	PSO1, PSO2
Bloom Level	L5 Evaluate
Skills Tested	File handling, CSV, exception handling, dictionaries, sorting, data cleaning

Problem Statement: A CSV file contains transaction_id, account_id, type, amount and timestamp. Create a program that reads the file, validates every row, writes valid credit and debit transactions into separate CSV files, writes rejected rows into error.csv with reason, and finally prints account-wise balance changes. The program must continue processing after invalid rows using exception handling.

Input Format: Path of input CSV file. The CSV file contains a header row followed by records.

Output Format: credit.csv, debit.csv, error.csv and console summary of account-wise net amount in descending order of absolute balance change.

Constraints: $1 \leq \text{rows} \leq 1000000$, amount > 0 , type is CREDIT or DEBIT, timestamp is ISO-like yyyy-mm-ddThh:mm:ss

Sample Input: transactions.csv containing:

```
tid,acc,type,amount,time
T1,A1,CREDIT,500,2026-01-01T10:00:00
T2,A1,DEBIT,abc,2026-01-01T10:05:00
T3,A2,DEBIT,100,2026-01-01T10:10:00
```

Sample Output: Console:

A1 500

A2 -100

Files created: credit.csv, debit.csv, error.csv

Sample Explanation: T2 is rejected because amount is not numeric. Other rows are processed and summarized.

Expected Time Complexity: $O(r \log a)$ where r is rows and a is number of accounts; $O(r)$ if no sorted output is required

Expected Space Complexity: $O(a)$

Q5. Object-Oriented Bank Settlement System

Field	Details
Difficulty	Product-Based Company Level
CO Mapping	CO-1, CO-2
PO Mapping	PO1, PO2, PO3, PO5, PO8, PO12
PSO Mapping	PSO1, PSO2, PSO3
Bloom Level	L6 Create
Skills Tested	OOP, encapsulation, custom exceptions, rollback logic, dictionaries

Problem Statement: Design an OOP-based settlement engine with Account, Transaction and Bank classes. The engine must support deposit, withdraw and transfer operations, maintain transaction history, prevent overdraft, and rollback an entire batch if any transaction in the batch fails. Use custom exceptions and suitable getters/setters or properties.

Input Format: Initial accounts followed by q operations. Operation format: DEPOSIT acc amount, WITHDRAW acc amount, TRANSFER from to amount, or BATCH_BEGIN/BATCH_END.

Output Format: Final balance of all accounts in account_id order, and FAILED batch number if any batch was rolled back.

Constraints: $1 \leq \text{accounts} \leq 100000$, $1 \leq q \leq 300000$, amount $\leq 10^9$, all balances fit in 64-bit integer

Sample Input: 3

A 1000

B 200

C 0

5

TRANSFER A B 300

BATCH_BEGIN

WITHDRAW B 1000

DEPOSIT C 50

BATCH_END

Sample Output: FAILED 1

A 700

B 500

C 0

Sample Explanation: The batch fails because B cannot withdraw 1000. The deposit to C is rolled back.

Expected Time Complexity: $O(q \log n)$ if ordered maps are used; $O(q + n \log n)$ for dictionary plus final sorting

Expected Space Complexity: $O(n + \text{batch_size})$

Q6. Python Module Dependency Resolver

Field	Details
Difficulty	FAANG Interview Level
CO Mapping	CO-1, CO-2
PO Mapping	PO1, PO2, PO3, PO5, PO12
PSO Mapping	PSO1, PSO2, PSO3
Bloom Level	L4 Analyze
Skills Tested	Graphs, topological sorting, heaps, sets, dictionaries, cycle detection

Problem Statement: Given a set of Python module names and import relationships, determine a valid loading order. If a circular dependency exists, print CYCLE and the modules involved in one cycle. Ignore duplicate import edges. If multiple modules can be loaded, choose lexicographically smallest module first.

Input Format: First line: n e. Next n lines: module name. Next e lines: module_a imports module_b.

Output Format: A lexicographically smallest valid module loading order, or CYCLE followed by one detected cycle.

Constraints: $1 \leq n \leq 200000$, $0 \leq e \leq 500000$, module names length ≤ 50

Sample Input: 4 3

core

db

ui

auth

ui auth

auth db

db core

Sample Output: core db auth ui

Sample Explanation: core has no dependency. db depends on core, auth depends on db and ui depends on auth.

Expected Time Complexity: $O((n+e) \log n)$

Expected Space Complexity: $O(n+e)$

Q7. Interactive Formula Validator with Custom Exceptions

Field	Details
Difficulty	Advanced University / Placement
CO Mapping	CO-1, CO-2
PO Mapping	PO1, PO2, PO5, PO8, PO10, PO12
PSO Mapping	PSO1, PSO2
Bloom Level	L5 Evaluate
Skills Tested	Exception handling, assertions, dictionaries, parsing, interactive programs

Problem Statement: Build an interactive calculator that accepts formulas of the form operand operator operand. Operands may be integers, decimals or previously stored variables. Supported operators are +, -, *, / and %. Invalid format, unknown variables, division by zero and unsupported operators must raise separate custom exceptions. The calculator must continue until the user enters quit.

Input Format: Multiple lines containing formulas, assignment statements such as $x = 10$, or quit.

Output Format: For each valid formula, print the result. For invalid lines, print the exception type and a meaningful message.

Constraints: Up to 100000 input lines; variable names follow Python identifier rules; values fit in float range

Sample Input: $x = 10$

$x + 5$

```
x / 0
3 ** 2
quit
```

Sample Output: 15
DivisionByZeroError
UnsupportedOperatorError

Sample Explanation: The second formula is valid. Division by zero and unsupported operator are reported without stopping the program.

Expected Time Complexity: $O(1)$ average per formula using dictionary lookup

Expected Space Complexity: $O(v)$ where v is number of variables

Q8. Compressed Log Index using Pickle and Zip

Field	Details
Difficulty	Advanced University / Industry
CO Mapping	CO-1, CO-4
PO Mapping	PO1, PO2, PO4, PO5, PO10, PO12
PSO Mapping	PSO1, PSO2, PSO3
Bloom Level	L6 Create
Skills Tested	File handling, pickling, zip/unzip, dictionaries, text processing, indexing

Problem Statement: A server produces several text log files. Create a program that scans all files in a folder, builds an inverted index mapping each normalized token to the list of file names and line numbers where it appears, stores the index using pickle, and compresses both original logs and index into a zip file. The program must also support a search mode that loads the pickle index and answers token queries.

Input Format: Mode BUILD or SEARCH. BUILD receives folder path and output zip name. SEARCH receives pickle path and q query tokens.

Output Format: In BUILD mode, print total files, total lines and unique tokens. In SEARCH mode, print all file:line matches for each token.

Constraints: $1 \leq \text{files} \leq 10000$, $\text{total lines} \leq 5000000$, $\text{token length} \leq 50$

Sample Input: BUILD logs archive.zip

Sample Output: FILES 3
LINES 1200
TOKENS 845

Sample Explanation: The index is built for all log files and compressed with the source data.

Expected Time Complexity: $O(\text{total characters in all files})$

Expected Space Complexity: $O(\text{unique tokens} + \text{postings})$

Q9. Threaded Job Scheduler Simulation

Field	Details
Difficulty	Product-Based Company Level
CO Mapping	CO-1, CO-2
PO Mapping	PO1, PO2, PO3, PO5, PO9, PO12
PSO Mapping	PSO1, PSO2, PSO3
Bloom Level	L5 Evaluate
Skills Tested	Multithreading concepts, priority queues, simulation, synchronization design

Problem Statement: Simulate a job scheduler where each job has `job_id`, priority, duration and required resource count. Multiple worker threads execute jobs. Higher priority jobs must be processed first; jobs with same priority use earlier arrival order. The program must maintain thread-safe job assignment and produce a final execution report.

Input Format: First line: w n . Next n lines: `arrival_time job_id priority duration resources`. Assume simulated time in integer units.

Output Format: For each job, print `job_id`, `worker_id`, `start_time` and `finish_time`. Finally print average waiting time.

Constraints: $1 \leq w \leq 64$, $1 \leq n \leq 200000$, $1 \leq \text{duration} \leq 10^6$, $1 \leq \text{resources} \leq 100$

Sample Input: 2 3
0 J1 2 5 1
1 J2 5 3 1
2 J3 2 2 1

Sample Output: J1 W1 0 5

J2 W2 1 4

J3 W2 4 6

AVG_WAIT 0.67

Sample Explanation: J2 has higher priority and is assigned when a worker is available. J3 waits until W2 becomes free.

Expected Time Complexity: $O(n \log n)$

Expected Space Complexity: $O(n + w)$

Q10. Tkinter Assignment Tracker with File Persistence

Field	Details
Difficulty	Advanced University / Mini Project
CO Mapping	CO-1, CO-4, CO-5
PO Mapping	PO1, PO2, PO3, PO5, PO9, PO10, PO12
PSO Mapping	PSO1, PSO2, PSO3
Bloom Level	L6 Create
Skills Tested	Tkinter, GUI design, event handling, file persistence, validation, CSV/JSON

Problem Statement: Create a Tkinter GUI application to manage student assignment submissions. The application must support adding a student, adding a submission, updating marks, filtering by pending/completed status, and exporting a CSV report. Data must persist locally using JSON or CSV files. Input fields must be validated before saving.

Input Format: GUI inputs through Entry, Text, Radio Button, Check Button, Listbox/Menu or Combobox widgets.

Output Format: Interactive GUI plus exported CSV report with enrollment, name, assignment, status, marks and remarks.

Constraints: At least 5 widgets must be used. The application must handle at least 5000 records without visible freezing during simple filtering.

Sample Input: GUI action: Add student 2201, Asha; Add Assignment 1 marks 18/20; Export CSV.

Sample Output: CSV row: 2201,Asha,Assignment 1,Completed,18,

Sample Explanation: The application stores the submission record and exports it in CSV format.

Expected Time Complexity: $O(r)$ for report generation; $O(\log r)$ or $O(r)$ acceptable for search depending on chosen structure

Expected Space Complexity: $O(r)$

Bloom Taxonomy and Question-Level CO Mapping

Q.No	Question Title	CO	PO	PSO	Bloom	Skills Tested
Q1	Campus Merit Analyzer using Compound Data Structures	CO-2, CO-4	PO1, PO2, PO5, PO10, PO12	PSO1, PSO2	L4 Analyze	Lists, tuples, dictionaries, sorting, hashing, data aggregation
Q2	Optimized Password Audit with Pattern Constraints	CO-1, CO-3	PO1, PO2, PO5, PO8, PO12	PSO1, PSO2	L5 Evaluate	Strings, regular expressions, hashing/trie, validation, edge-case analysis
Q3	Recursive Expression Engine with Memoization	CO-1, CO-2	PO1, PO2, PO3, PO5, PO12	PSO1, PSO2, PSO3	L6 Create	Recursion, stacks, dictionaries, parsing, memoization, cycle detection
Q4	Exception-Safe CSV Transaction Splitter	CO-1, CO-4	PO1, PO2, PO5, PO8, PO10, PO12	PSO1, PSO2	L5 Evaluate	File handling, CSV, exception handling, dictionaries, sorting, data cleaning
Q5	Object-Oriented Bank Settlement System	CO-1, CO-2	PO1, PO2, PO3, PO5, PO8, PO12	PSO1, PSO2, PSO3	L6 Create	OOP, encapsulation, custom exceptions, rollback logic, dictionaries
Q6	Python Module Dependency Resolver	CO-1, CO-2	PO1, PO2, PO3, PO5, PO12	PSO1, PSO2, PSO3	L4 Analyze	Graphs, topological sorting, heaps, sets, dictionaries, cycle detection
Q7	Interactive Formula Validator with Custom Exceptions	CO-1, CO-2	PO1, PO2, PO5, PO8, PO10, PO12	PSO1, PSO2	L5 Evaluate	Exception handling, assertions, dictionaries, parsing, interactive programs
Q8	Compressed Log Index using Pickle and Zip	CO-1, CO-4	PO1, PO2, PO4, PO5, PO10, PO12	PSO1, PSO2, PSO3	L6 Create	File handling, pickling, zip/unzip, dictionaries, text processing, indexing
Q9	Threaded Job Scheduler Simulation	CO-1, CO-2	PO1, PO2, PO3, PO5, PO9, PO12	PSO1, PSO2, PSO3	L5 Evaluate	Multithreading concepts, priority queues, simulation, synchronization design
Q10	Tkinter Assignment Tracker with File Persistence	CO-1, CO-4, CO-5	PO1, PO2, PO3, PO5, PO9, PO10, PO12	PSO1, PSO2, PSO3	L6 Create	Tkinter, GUI design, event handling, file persistence, validation, CSV/JSON

CO-PO-PSO Mapping Matrix for NAAC

Mapping scale: 3 = High, 2 = Moderate, 1 = Low, blank = No direct mapping.

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO-1	3	2	2		3			2		1		2	3	3	2
CO-2	3	3	2		2							2	3	2	2
CO-3	2	3	2		3			2				2	2	3	2
CO-4	3	3	2	2	3					2	1	2	3	3	3
CO-5	2	2	3		3	1		1	2	2		2	2	3	3

Short PO/PSO Code Reference

Code	Short Meaning
PO1	Engineering knowledge
PO2	Problem analysis
PO3	Design/development of solutions
PO4	Investigation
PO5	Modern tool usage
PO6	Engineer and society
PO7	Environment and sustainability
PO8	Ethics
PO9	Individual/team work
PO10	Communication
PO11	Project management

Code	Short Meaning
PO12	Life-long learning
PSO1	Core programming and database knowledge
PSO2	Tools and technologies
PSO3	Innovation and development

CO Mapping Justification

CO	Mapped PO/PSO Justification
CO-1	Students implement, test, debug and optimize Python programs using modern tools, exception-safe logic and documented coding practices. Mapped outcomes: PO1, PO2, PO3, PO5, PO8, PO10, PO12; PSO1, PSO2, PSO3.
CO-2	Students solve algorithmic problems with functions, recursion, lists, tuples, dictionaries and modular design for scalable problem solving. Mapped outcomes: PO1, PO2, PO3, PO5, PO12; PSO1, PSO2, PSO3.
CO-3	Students analyze and validate text using regular expressions for practical filtering, extraction, security checks and data cleaning tasks. Mapped outcomes: PO1, PO2, PO3, PO5, PO8, PO12; PSO1, PSO2, PSO3.
CO-4	Students manipulate structured data, files, CSV/JSON/database results and prepare outputs suitable for analysis, visualization and reporting. Mapped outcomes: PO1, PO2, PO3, PO4, PO5, PO10, PO11, PO12; PSO1, PSO2, PSO3.
CO-5	Students design GUI applications using Tkinter and integrate event-driven interfaces with files, databases and validation logic. Mapped outcomes: PO1, PO2, PO3, PO5, PO6, PO8, PO9, PO10, PO12; PSO1, PSO2, PSO3.

Evaluation Rubric

Criterion	Weightage	Assessment Focus
Problem Understanding	20%	Correct interpretation of input/output, constraints, edge cases and required features.
Algorithm Design	25%	Use of efficient data structures, algorithmic strategy, modular functions and clear reasoning.
Code Correctness	20%	Produces correct output for sample, hidden, boundary and invalid test cases.
Optimization	15%	Meets expected time and space complexity; avoids unnecessary nested loops and repeated work.
Code Quality	10%	Readable naming, proper decomposition, reusable modules/classes and Pythonic structure.
Testing	5%	Includes meaningful test cases covering normal, edge and failure scenarios.
Documentation	5%	Clear comments, assumptions, run instructions and complexity notes.

Learning Outcomes After Completing the Assignment

- Develop optimized Python programs for complex and multi-step programming problems.
- Apply functions, recursion, OOP, file handling, regular expressions, database access and GUI concepts in practical tasks.
- Analyze time and space complexity before finalizing a solution.
- Strengthen coding interview readiness through constraints, edge cases and algorithmic reasoning.
- Design maintainable, testable and well-documented Python solutions.
- Use modern programming tools and debugging methods for reliable software development.